- *Public endpoints*—Identify all public endpoints that external users can access. These are the primary entry points into your application and are often targeted by malicious actors.
- *User input points*—Determine where user input is accepted, such as forms or API requests. Understanding these points will help you focus on areas that require stringent validation.
- *Data sensitivity*—Evaluate the sensitivity of the data being processed. Inputs that could lead to exposure of sensitive information should be prioritized in your validation strategy. Important personal details like emails, passwords, IPs, or special identifiers are all classified as personal data.

Take, for example, our React server–component Next.js applications, where we often invoke POST requests to the current page at the following request URL:

```
http://localhost:3000/
```

In contrast, when we did not use React server components (in chapter 3, for example), the endpoint typically resided in the API routes at

```
http://localhost:3000/api
```

It is crucial to monitor all public endpoints, since they represent the main entry points for your applications. Once you do that, you can start securing all data that flows in or out of the application.

SERVER VS. CLIENT VALIDATION

When implementing input validation, it's important to distinguish between server and client validation. Here are their main differences:

- *Server validation*—This occurs on the server after data has been submitted. It is important because it ensures that even if client validation is bypassed (which can happen easily), the server still checks for valid input before processing it. Server validation should always be performed on each client request.
- *Client validation*—This occurs in the browser before data is sent to the server. While it can enhance user experience by providing immediate feedback, it should never be relied on as the sole method of validation. Malicious users can manipulate client code and bypass these checks.

Always assume that any data coming from the client can be tampered with: server validation must be the source of truth.

EXAMPLE: INPUT VALIDATION OF NEXT.JS API HANDLER

As an example, say we want to filter the user input prompt so it will not exceed 1,000 characters in length. We can implement Zod within the API handler.